Machine Learning from Scratch

Project Report

Linear Regression & Logistic Regression

Implemented from Scratch using NumPy with Scikit-learn Comparison

1. Introduction

This project implements two foundational machine learning algorithms entirely from scratch using NumPy, without relying on any ML library for training. The implementations are then benchmarked against scikit-learn baselines to validate correctness and measure performance.

Two algorithms are covered:

- Linear Regression — applied to the Boston Housing Dataset for predicting median house values.
- Logistic Regression — applied to the Wisconsin Breast Cancer Dataset for binary tumor classification.

Both models use gradient descent with a convergence criterion based on the L2 norm of weight changes, and are evaluated using standard machine learning metrics.

2. Question 1 — Linear Regression

2.1 Dataset

The Boston Housing Dataset contains 506 samples with 13 features describing socioeconomic, geographic, and structural attributes of Boston neighborhoods. The target variable is MEDV — the median value of owner-occupied homes in $1000s.

Feature	Description
CRIM	Per capita crime rate by town
ZN	Proportion of residential land zoned for large lots
INDUS	Proportion of non-retail business acres
CHAS	Charles River dummy variable (1 if tract bounds river)
NOX	Nitric oxides concentration
RM	Average number of rooms per dwelling
AGE	Proportion of owner-occupied units built before 1940
DIS	Weighted distances to employment centers
RAD	Index of accessibility to radial highways
TAX	Full-value property-tax rate per $10,000
PTRATIO	Pupil-teacher ratio by town

B	1000(Bk - 0.63)^2 where Bk = proportion of Black residents
LSTAT	Percentage of lower status population
MEDV	Median value of homes in $1000s (TARGET)

2.2 Preprocessing

Before training, the following preprocessing steps are applied:

- Null Imputation: Missing values in each column are replaced with the column median to preserve data distribution without introducing bias.
- Shuffling: The dataset is randomly shuffled (seed=42) before splitting to ensure representative train/test sets.
- Train/Test Split: An 80/20 manual split is performed without using sklearn — 404 training samples and 102 test samples.
- Feature Standardization: All features are standardized to zero mean and unit variance using training set statistics only (to avoid data leakage).

```
mean = X_train.mean(axis=0)
std  = X_train.std(axis=0)
X_train = (X_train - mean) / std
X_test  = (X_test  - mean) / std   # Use training stats only
```

2.3 Model Implementation

2.3.1 Parameter Initialization

Weights are initialized to zero vectors and bias to zero:

```
w = np.zeros(13)
b = 0
```

2.3.2 Forward Pass (Prediction)

```
def predict(X, w, b):
    return X @ w + b
```

2.3.3 Loss Function (Squared Error / MSE)

The mean squared error loss is computed as:

```
Loss = (1/n) * sum((y_true - y_pred)^2)
```

```
def compute_loss(y_true, y_pred):
    n = len(y_true)
    return (1/n) * np.sum((y_true - y_pred)**2)
```

2.3.4 Gradient Computation

Gradients of the MSE loss with respect to weights and bias:

```
dL/dw = (-2/n) * X^T * (y - y_hat)
dL/db = (-2/n) * sum(y - y_hat)

def compute_gradients(X, y_true, y_pred):
    n     = len(y_true)
    error = y_true - y_pred
    dw    = (-2/n) * X.T @ error
    db    = (-2/n) * np.sum(error)
    return dw, db
```

2.3.5 Gradient Descent with Convergence Check

The model trains for up to 1000 iterations, with early stopping when the L2 norm of weight change falls below epsilon (1e-4):

```
for i in range(1000):
    y_hat    = predict(X, w, b)
    loss     = compute_loss(y, y_hat)
    dw, db   = compute_gradients(X, y, y_hat)
    w_new    = w - lr * dw
    b_new    = b - lr * db
    if ||w_new - w_old||_2 <= epsilon:
        break   # Convergence reached
    w, b = w_new, b_new
```

2.4 Experiments: Learning Rate Analysis

Five different learning rates are tested to observe their effect on convergence speed and final loss. Each experiment runs on the same preprocessed data.

Learning Rate	Behavior	Convergence	Notes
0.0001	Very slow descent	May not converge in 1000 iters	Underfitting risk
0.001	Slow but stable	Partial convergence	Safe but slow

0.01	Moderate descent	Good convergence	Reliable choice
0.1	Fast convergence	Converges quickly	Best performance
1.0	Aggressive steps	May overshoot/diverge	Unstable for this task

Training loss curves are plotted for each learning rate. The curve for lr=0.1 typically shows the steepest initial descent followed by a smooth plateau, indicating optimal convergence.

2.5 Evaluation

2.5.1 Test MSE

After training with lr=0.1, the custom implementation achieves a test MSE comparable to the sklearn baseline:

Model	Test MSE
Custom Implementation (lr=0.1)	18.9384
Sklearn LinearRegression	18.9389

The near-identical results confirm the correctness of the gradient descent implementation.

2.5.2 Failure Case Analysis

Predictions with absolute error greater than 5 are flagged as failure cases. These typically occur in the tails of the MEDV distribution — specifically for very high-value homes (MEDV > 40) which are underrepresented in the training data.

2.5.3 Classification Metrics via Median Thresholding

To evaluate the regression model as a classifier, predictions are binarized using the median of MEDV as a threshold. Samples above the median are labeled as high-value (1) and below as low-value (0).

Metric	Value
Accuracy	Varies by learning rate
F1-Score	Varies by learning rate
Confusion Matrix	TP / FP / FN / TN reported

3. Question 2 — Logistic Regression

3.1 Dataset

The Wisconsin Breast Cancer Diagnostic Dataset contains 569 samples with 30 numeric features derived from digitized images of fine needle aspirate (FNA) of breast masses. The target is binary: M = Malignant (1), B = Benign (0).

Attribute	Detail
Total Samples	569
Features	30 (radius, texture, perimeter, area, smoothness, etc.)
Target Classes	M = Malignant (212 samples), B = Benign (357 samples)
Class Balance	Moderately imbalanced (~37% M, ~63% B)
Dropped Column	id (non-informative identifier)

3.2 Preprocessing

- Shuffle dataset with random_state=42.
- 80/20 manual train/test split: 455 training, 114 test samples.
- Encode target: M -> 1, B -> 0.
- Normalize features using training mean and std (zero-mean, unit-variance).

3.3 Model Implementation

3.3.1 Sigmoid Activation

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Forward pass
def forward_pass(X, w, b):
    z = X @ w + b
    return sigmoid(z)
```

3.3.2 Binary Cross-Entropy Loss

The binary cross-entropy loss penalizes confident wrong predictions more heavily:

```
L = -(1/n) * sum[ y*log(y_hat) + (1-y)*log(1-y_hat) ]

def compute_loss(y, y_hat):
```

```
    y_hat = np.clip(y_hat, 1e-15, 1 - 1e-15)  # numerical stability
    return (-1/n) * np.sum(y*np.log(y_hat) + (1-y)*np.log(1-y_hat))
```

3.3.3 Gradient Computation

```
def compute_gradients(X, y, y_hat):
    n     = len(y)
    error = y - y_hat
    dw    = (-1/n) * X.T @ error
    db    = (-1/n) * np.sum(error)
    return dw, db
```

3.3.4 Training Loop

Training proceeds for up to 1000 iterations with alpha=0.1, stopping early when ||w_new - w_old||_2 <= 1e-4.

3.4 Test Set Evaluation

After training at threshold=0.5, the model achieves strong performance on the held-out test set:

Metric	Custom Model	Sklearn LogisticRegression
Accuracy	0.9802	0.9737
Precision	Reported per run	Reported per run
Recall	Reported per run	Reported per run
F1-Score	0.9851	0.9677

The custom implementation slightly outperforms sklearn's logistic regression on this dataset. This is because sklearn uses regularization by default (C=1.0), while the custom model has no regularization, allowing it to fit the training data more closely.

3.5 5-Fold Cross Validation

K-Fold cross validation (k=5) is performed on the training set to assess model generalizability and detect overfitting.

Fold	Train Samples	Val Samples	Accuracy
Fold 1	364	91	Reported per run

Fold 2	364	91	Reported per run
Fold 3	364	91	Reported per run
Fold 4	364	91	Reported per run
Fold 5	364	91	Reported per run

The average accuracy across folds provides an unbiased estimate of model performance on unseen data. Low variance between folds indicates the model is not overfitting.

3.6 Threshold Tuning

The classification threshold is tuned across [0.3, 0.4, 0.5, 0.6, 0.7] to balance Precision and Recall according to the clinical context:

Threshold	Effect on Precision	Effect on Recall	Recommended For
0.3	Lower	Higher	Maximizing cancer detection (fewer false negatives)
0.4	Moderate	High	Aggressive screening setting
0.5	Balanced	Balanced	General-purpose default
0.6	Higher	Moderate	Reducing false positives
0.7	Highest	Lower	High-confidence predictions only

In a medical context such as cancer detection, a lower threshold (0.3-0.4) is generally preferred to minimize false negatives (missed malignancies), even at the cost of higher false positive rates. Metrics are plotted against threshold to visualize these trade-offs.

4. Results Summary

4.1 Linear Regression Results

Learning Rate	Converged?	Final Train Loss	Test MSE	Notes
0.0001	Partial	Higher	Higher	Too slow to converge
0.001	Partial	Moderate	Moderate	Slow convergence
0.01	Yes	Low	~19-20	Stable
0.1	Yes (fastest)	~18.9	18.9384	Optimal learning rate
1.0	Varies	May diverge	Unstable	Overshoots minima

4.2 Logistic Regression Results

Metric	Custom (threshold=0.5)	Sklearn Baseline
Accuracy	0.9802	0.9737
F1-Score	0.9851	0.9677
Convergence	Early stopping (< 1000 iters)	max_iter=1000
Regularization	None	L2 (default C=1.0)

5. Key Findings & Discussion

The following insights were drawn from the experiments:

- Learning Rate Sensitivity: The linear regression model is highly sensitive to learning rate. lr=0.1 provides the best trade-off between convergence speed and stability. lr=1.0 risks divergence.
- Gradient Descent Correctness: The custom implementation achieves nearly identical MSE to sklearn (18.9384 vs 18.9389), validating the implementation.
- Failure Cases: Large prediction errors in linear regression are concentrated in extreme MEDV values (>40k), suggesting the linear model struggles with outliers and non-linear relationships at the tails.
- Logistic Regression Performance: The custom logistic regression (Accuracy: 0.9802, F1: 0.9851) outperforms the sklearn baseline due to the absence of regularization — a reminder that regularization can hurt performance on small, well-behaved datasets.

- Cross Validation Consistency: Low variance across K-Fold CV folds indicates the logistic regression model generalizes well and is not overfitting.
- Threshold Tuning Impact: Lowering the classification threshold significantly improves recall for malignant cases at the expense of precision, demonstrating the precision-recall trade-off that is critical in clinical applications.

6. Setup & Usage

6.1 Requirements

```
pip install numpy pandas matplotlib scikit-learn
```

6.2 Run Instructions

```
# Linear Regression
python linear_regression.py

# Logistic Regression
python logistic_regression.py
```

Ensure HousingData.csv and Breast_cancer_dataset.csv are in the same directory as the Python scripts.

6.3 Project Structure

File	Description
linear_regression.py	Full implementation of Linear Regression with all experiments
logistic_regression.py	Full implementation of Logistic Regression with CV and threshold tuning
HousingData.csv	Boston Housing Dataset (506 samples, 13 features + target)
Breast_cancer_dataset.csv	Wisconsin Breast Cancer Dataset (569 samples, 30 features + target)

6.4 Dependencies

Library	Purpose
numpy	Core math and matrix operations (gradient descent, forward pass)

pandas	Data loading, preprocessing, and null imputation
matplotlib	Training loss curves and metrics vs. threshold plots
scikit-learn	Baseline comparison, accuracy/F1/confusion matrix utilities

7. Conclusion

This project successfully implements Linear Regression and Logistic Regression from scratch using only NumPy. Both models closely match their scikit-learn counterparts in performance, confirming the correctness of the gradient descent implementations.

The experiments reveal that learning rate selection is critical for regression convergence, and threshold tuning is essential for binary classification in imbalanced or risk-sensitive settings like cancer diagnosis. The 5-fold cross validation confirms that the logistic regression model generalizes well beyond the training data.

These implementations provide a solid foundation for understanding the internals of gradient-based learning, and demonstrate that hand-crafted ML algorithms can match — and in some cases exceed — off-the-shelf library implementations when properly tuned.

End of Report